

# serial\_bridge から ros2can へ

---

serial\_bridge を使ってきた人向け ros2can 紹介・移行ガイド

立命館大学 ロボット技術研究会 RRST NHK プロジェクト

# ひとことで言うと

## 配線

serial\_bridge

マイコンごとに USB で PC へ

ros2can

CAN のデジーチェーンで PC とは USB1 本

## 操作

ターミナルのログ表示のみ

GUI で送信・確認・FW 書き込み

## 互換

serial\_tx\_[ID] / serial\_rx\_[ID]

トピック・フレーム形式はそのまま

# 01

## なぜ **ros2can** を作ったのか

serial\_bridge の運用で見えてきたこと

# 開発背景



## serial\_bridge で見えてきた課題

### 配線が増える

マイコン 1 台につき USB1 本。  
台数が増えるほどケーブルと  
USB ポートが増える

### 状態が見えにくい

ターミナルのログのみ。指令送信  
やパラメータ確認は別ツールが  
必要

### 専用アクチュエータ

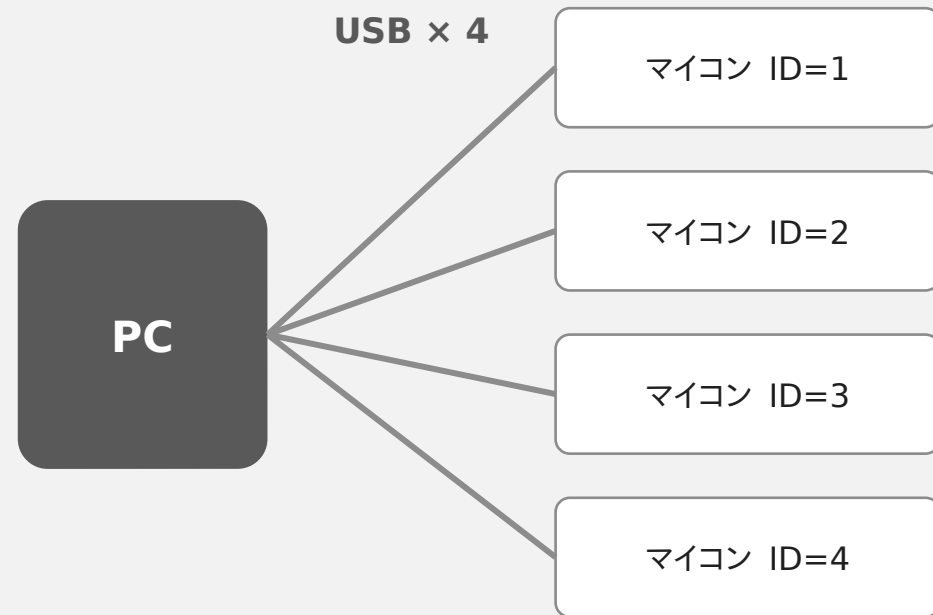
ロボマス・CubeMars を汎用出  
力の延長でしか扱えない

### 実機がないと試せない

マイコンを繋がないと、ノードやト  
ピック連携を確認できない

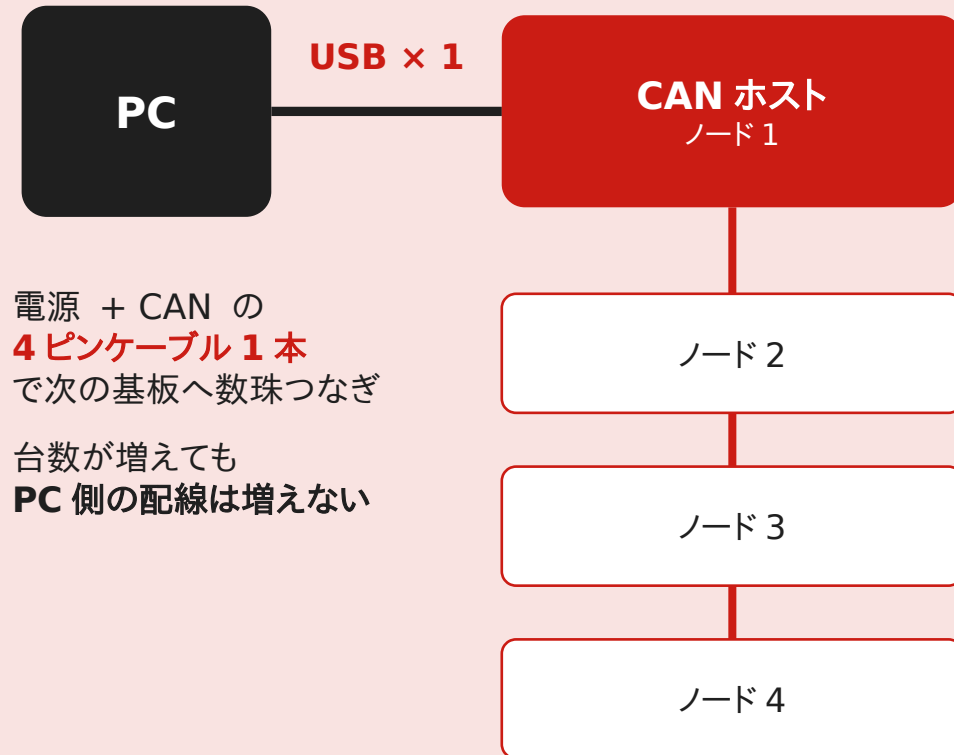
# 最大の目玉:配線が USB1 本になる

## serial\_bridge : 星型配線



マイコンの台数分の USB ケーブルとポートが必要。台数が増えるほど PC 周りの配線が増える

## ros2can : デイジーチェーン



# 数字で見る ros2can

**4 → 1**

PC との USB ケーブル  
( 4 基板構成の場合)

**最大 4**

1 バスあたりのノード数  
(ホストを含む)

**1 Mbps**

CAN バスの通信速度  
(ロボマス等と共通)

**4 ピン**

電源 +CAN を 1 本の  
ケーブルで次の基板へ

## USB が減ると何がうれしいか

- PC の USB ポートやハブの数に縛られにくくなる
- 機体内の長い USB ケーブルの取り回しが減り、配線がシンプルになる
- 基板を増やすときは、デイジーチェーンの末尾に足すだけ

# 02

## 何が変わるのか

比較表と新機能

# serial\_bridge と ros2can の比較

| 観点              | serial_bridge                    | ros2can                                      |
|-----------------|----------------------------------|----------------------------------------------|
| 配線              | マイコン 1 台 = USB1 本の星型             | CAN ホスト ( USB1 本 ) + CAN デイジーチェーン            |
| UI              | ターミナルのみ ( テキスト / ASCII バー / なし ) | PyQt5 GUI ( Control / Monitor / Raw / Info ) |
| スロットの意味         | FW ごとに固定 ( 1 ~ 8 = DC モータ 等 )    | プロファイルで可変、CAN ノードへ分配                         |
| エンコーダ           | 生のパルス値をそのまま Publish              | オーバーフロー展開・原点セット ( _unwrapped / _zeroed )     |
| 実機なしの確認         | なし                               | 仮想デバイスで確認できる                                 |
| ロボマス / CubeMars | 汎用のロボマスモードのみ                     | 専用ドライバ ( MIT モード等 )                          |
| FW の設定          | config.hpp を手で編集                 | GUI で生成・書き込み                                 |
| ポート専有           | 排他フラグなし                          | TIOCEXCL で排他専有                               |
| 実装              | C++ ( rclcpp )                   | Python ( rclpy + PyQt5 )                     |

# ros2can の新機能

## 1 GUI で直接操作

4 タブで指令送信とセンサ確認。ダイレクト送信は既定 OFF

## 2 仮想デバイス

実機なしで UI・トピック連携を確認。rqt や自作ノードのテストにも

## 3 エンコーダ処理

オーバーフロー展開と原点セット。  
\_unwrapped / \_zeroed を配信

## 4 ロボマス・CubeMars

専用モードで直接制御。MIT モードで Kp/Kd を毎周期変更

## 5 FW 生成・書き込み

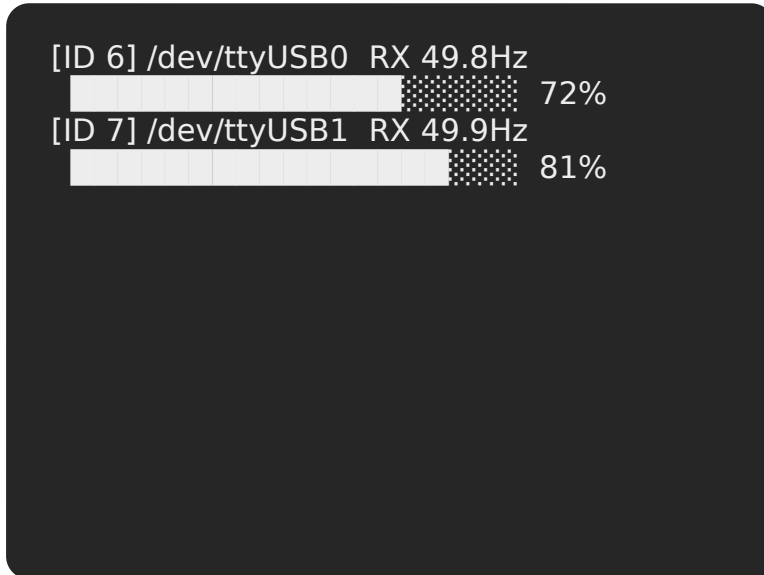
設定を GUI で入力 →  
generated\_firmware に生成 → そのまま書き込み

## 6 安全機能

全デバイス E-STOP、閉じるとゼロ指令、全ゼロで安全停止

# 状態確認:ターミナルから GUI へ

## serial\_bridge



- ログやバーグラフで通信状態がわかる
- 値の確認・指令送信は `ros2 topic` など別ツール

※ 表示はイメージ

## ros2can



指令送信・センサ値・生データ・通信状態をすべて **GUI** で確認

# 03

## 何が変わらないのか・どう移行するか

トピック互換と、気をつけるポイント

# そのまま使えるもの

**トピック名** serial\_tx\_[ID] / serial\_rx\_[ID]

**メッセージ型** std\_msgs/msg/Int16MultiArray

**データ長** 24 × int16

**フレーム形式** 0xAA | ID | LEN | DATA | XOR

**ID 方式** DEVICE\_ID で識別（ポート順に非依存）

**スキャン** 自動スキャン・ホットプラグ・  
excluded\_ports

→ 既存ノードの「**publish / subscribe** する部分」はほぼそのまま流用できます。

変わるのは「どのスロットが何を意味するか」だけ（次のスライド）

# 注意: スロットの意味が変わる

**serial\_bridge ( esp32\_serial\_bridge の例): FW ごとに固定**



**ros2can (既定プロファイル): ノードごとに 5 スロット**



## インデックス直書きは要修正

例: serial\_bridge の data[9] は SERVO1 だが、ros2can ではノード 2 の予備スロット(受信では ENC2 )

## ラッパーで吸収する

PacketController → Ros2CanPacketController  
 $global\ index = node \times 5 + local$  をライブラリが計算

# コードの移行例

## serial\_bridge

```
#include "common/PacketController.hpp"
PacketController pkt;

pub_ = create_publisher<Int16MultiArray>(
    "serial_tx_3", 10);

pkt.setServo(SERVO2, 90);
msg.data = pkt.toVector();

// 受信:インデックスを直接読む
int16_t enc1 = m->data[1];
bool sw1 = m->data[9];
```



## ros2can

```
#include "common/Ros2CanPacketController.hpp"
Ros2CanPacketController pkt;

pub_ = create_publisher<Int16MultiArray>(
    "serial_tx_1", 10); // 同じ

pkt.setServo(/*node*/0, 2, 90); // ノード番号が増える
msg.data = pkt.toVector(); // 同じ

// 受信:ラッパー経由
pkt.updateRx(m->data);
int16_t enc1 = pkt.getEnc(0, 1);
bool sw1 = pkt.getSW(0, 1);
```

サンプル: ros2can\_example ( servo\_sweep\_example / cubemars\_position\_example )、PS4 での操作例は  
cr26\_soki/ros2can\_practice.cpp

# 移行の手順

1

## **ros2can** をビルドして起動

`sudo apt install python3-pyqt5 python3-serial → colcon build`

2

## 仮想デバイスで既存ノードを確認

「デバッグデバイスを追加」→ 自作ノードの `publish / subscribe` を確認

3

## ノードのスロット処理を書き換え

`PacketController` → `Ros2CanPacketController` (インデックス直書きを解消)

4

## **FW** を生成して基板に書き込む

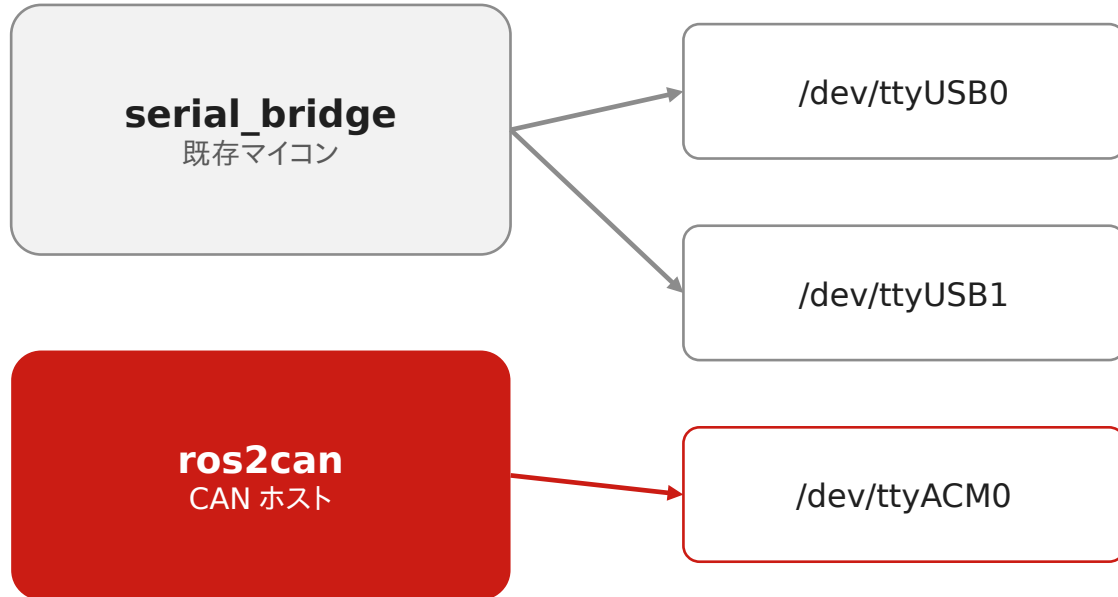
ホストは `MODE_CAN_HOST`、子は `MODE_CAN`。CAN\_ID は 101 ~ 104

5

## 配線してテスト

USB はホストだけ。バス両端の終端抵抗を確認 → ダイレクト送信 OFF のまま疎通確認

# 移行期間中の併用



- ros2can は **TIOCEXCL** でポートを排他専有
- ros2can が先に掴めば、serial\_bridge 側は open に失敗してリトライするだけ
- 逆 (serial\_bridge が先) は防げない

## 対策: **excluded\_ports** で分ける

```
# config/ros2can.yaml
excluded_ports: ["/dev/ttyUSB0",
                "/dev/ttyUSB1"]
```

```
# config/serial_bridge.yaml
excluded_ports: ["/dev/ttyACM0"]
```

お互いに相手が管理するポートを除外しておく、取り合いが起きません。

# 移行時に気をつけること

!

## フェイルセーフは **FW** に無い

通信が途切れてもマイコンは最後の指令を保持。終了時のゼロ指令 ( on\_shutdown ) は必須

2

## ダイレクト送信とトピック通過

ダイレクト送信 ON のデバイスは外部ノードの serial\_tx\_[ID] を無視する

3

## ピン共有の設定

MULTIn (サーボ /SW) ・ ENCn\_MD ( ENC/MD ) は config.hpp で固定。実配線と一致させる

4

## 終端抵抗と **CAN\_ID**

バスの両端に 120Ω。CAN\_ID ( 101 ~ 104、下 2 桁=ノード番号) を重複させない

5

## DC モータの数

既定プロファイル ( SOKI 基板) は DC モータ非搭載。MD が必要なら ENCn\_MD=1 か MES 基板

# まとめ

配線がシンプルに

USB1 本+ CAN デイジーチェーン

操作・確認が **GUI** で

仮想デバイス・FW 生成・E-STOP まで GUI で  
完結

移行は段階的に

トピック互換。スロットの意味だけ  
Ros2CanPacketController で吸収

新規の機体では **ros2can** の採用を強く推奨します。

## 参考資料

**ros2can** マニュアル

[rrst-nhk-project.github.io/ros2can/](https://rrst-nhk-project.github.io/ros2can/)

**serial\_bridge** との比較( **1.1** 節)

マニュアル 1.1 / 技術マニュアル 9.9

**serial\_bridge** との併用( **3.1** 節)

マニュアル 3.1

**GitHub**

[github.com/RRST-NHK-Project/ros2can](https://github.com/RRST-NHK-Project/ros2can)